

Project Description

A Tkinter PictureBox widget for displaying an OpenCV image.

This package assumes that either the **opencv-python** or the **opencv-contrib-python** package is currently installed in the user's development environment, and that the user is reasonably familiar with the OpenCV library.

Although OpenCV provides the **cv2.namedWindow()** and **cv2.imshow()** methods for displaying an OpenCV image in its own desktop window, this approach does not lend itself to displaying an OpenCV image inside the framework of a user created Tkinter window based application.

This package adds a fully compatible **PictureBox** widget to Tkinter's set of graphical user interface widgets, thereby providing the user with a convenient tool for displaying an OpenCV image in any Tkinter window based application.

Installation

```
pip install opencv-tk
```

API Documentation

This package provides the following class definitions :

- **PictureBox** - A Tkinter widget class for displaying an OpenCV image
- **BorderStyle** - A data class of the available border style options
- **ImageMode** - A data class of the available image display mode options

PictureBox

`PictureBox( parent, width, height )`

- ***parent* : Any** - The parent widget of the PictureBox.
- ***width* : int** - The initial width of the PictureBox. (in pixels)
- ***height* : int** - The initial height of the PictureBox. (in pixels)

By default, the **border_style** property is set to **BorderStyle.Flat**, and the **image_mode** property is set to **ImageMode.Normal**.

Properties

All of the PictureBox properties have read and write permissions.

- **border_style : str** - The border style of the PictureBox. This property can be set to any one of the available BorderStyle options (**e.g. BorderStyle.Ridge**).
- **image_mode : int** - The image display mode of the PictureBox. This property can be set to any one of the available ImageMode options (**e.g. ImageMode.Zoom**).
- **height : int** - The current height of the PictureBox. (in pixels)
- **width : int** - The current width of the PictureBox. (in pixels)

Methods

- **clear()** : Clear the currently displayed image.
- **display(image) -> bool** : Display the OpenCV image in the PictureBox. Returns **True** if the specified image is valid, **False** otherwise
 - **image : numpy.ndarray | cv2.Mat** - Images with a color space representation of Grayscale, BGR, or BGRA will be displayed correctly in the PictureBox. The **cv2.cvtColor(src, code)** function should be used to convert images, with other color spaces, into the BGR color space.
- **load(filename) -> bool** : Load and display an image from a file. Returns **True** if the image was successfully loaded and displayed, **False** otherwise.
 - **filename : str** - The full pathname of the image file.
- **save(filename) -> bool** : Save the currently displayed image to a file. Returns **True** if the image was successfully saved, **False** otherwise.
 - **filename : str** - The full pathname of the image file.
- **set_background(color)** - Set the background color of the PictureBox. The **color** parameter value can be provided in one of the following forms :
 - It can be expressed as a named color string, e.g. **'white'**. A defined set of named colors can be found at : [CSS Color 4: Named Colors](#)
 - It can be expressed as a 6-digit hexadecimal notation color string **'#rrggbb'**
 - It can be expressed as a tuple color value (**red, green, blue**) that conforms to one of two formats :
 - Three integer color components ranging in value from 0 to 255
 - Three floating point color components ranging in value from 0.0 to 1.0

BorderStyle

These are the available border style options for the PictureBox.

- **Raised** - The PictureBox appears raised above the background.
- **Sunken** - The PictureBox appears recessed into the background.
- **Groove** - The PictureBox has a carved groove border.
- **Ridge** - The PictureBox has a raised ridge border.
- **Solid** - The PictureBox has a simple solid border.
- **Flat** - The PictureBox appears flat, no border.

ImageMode

These are the available image display mode options for the PictureBox.

- **Normal** - The image is placed in the upper-left corner of the PictureBox. The image is clipped if it is larger than the PictureBox.
- **CenterImage** - If the PictureBox is larger than the image, the image is displayed in the center of the PictureBox. If the image is larger than the PictureBox, the image is centered in the PictureBox, and the outside edges of the image are clipped.
- **StretchImage** - The image is stretched or shrunk to fit the size of the PictureBox.
- **AutoSize** - The size of PictureBox is changed to match the size of the image.
- **Zoom** - The size of the image is increased or decreased to fit the size of the PictureBox, while keeping the image's original size ratio.

User Notes

The PictureBox widget is derived from the Tkinter Label widget. This means that the PictureBox inherits all of the Universal Tkinter widget methods, and that all of these methods are available to the user. This also means that all of the Label widget's options are exposed to the user. To avoid the possibility of creating any unpredictable PictureBox behavior, the user should never directly modify the values of the underlying Label widget's options. The one exception to this "hands-off rule" is that the user can safely modify the widget's **'state'** value.

PictureBox Usage Example

This example creates a Tkinter based window application that uses a **PictureBox** to display the live video from a webcam. The example uses the **cv2.VideoCapture** class to provide the live video from the webcam, and it continuously reads and displays new video frames until the user closes the application.

```
import tkinter as tk
import cv2
from opencv_tk import PictureBox, BorderStyle, ImageMode

class DemoWindow(tk.Frame):
    """The PictureBox demo window."""

    def __init__(self, root, camera):
        """Construct the PictureBox demo window."""
        super().__init__(root, bd=3, relief='ridge')
        root.resizable(False, False)
        root.title('PictureBox Demo')
        self.pack()

        self._picture_box = PictureBox(self, 400, 300)
        self._picture_box.border_style = BorderStyle.Sunken
        self._picture_box.image_mode = ImageMode.StretchImage
        self._picture_box.pack(padx=10, pady=10)

        self._camera = camera
        self._display_video()

    def _display_video(self):
        """Continuously read and display new video frames."""
        success, frame = self._camera.read()
        if success:
            self._picture_box.display(frame)
            self.after(10, self._display_video)

if __name__ == '__main__':
    webcam = cv2.VideoCapture(0)
    if webcam.isOpened():
        demo_window = DemoWindow(tk.Tk(), webcam)
        demo_window.mainloop()
        webcam.release()
    else:
        print('No webcam was detected!')
```

